

# Skiplists

COMP 2402

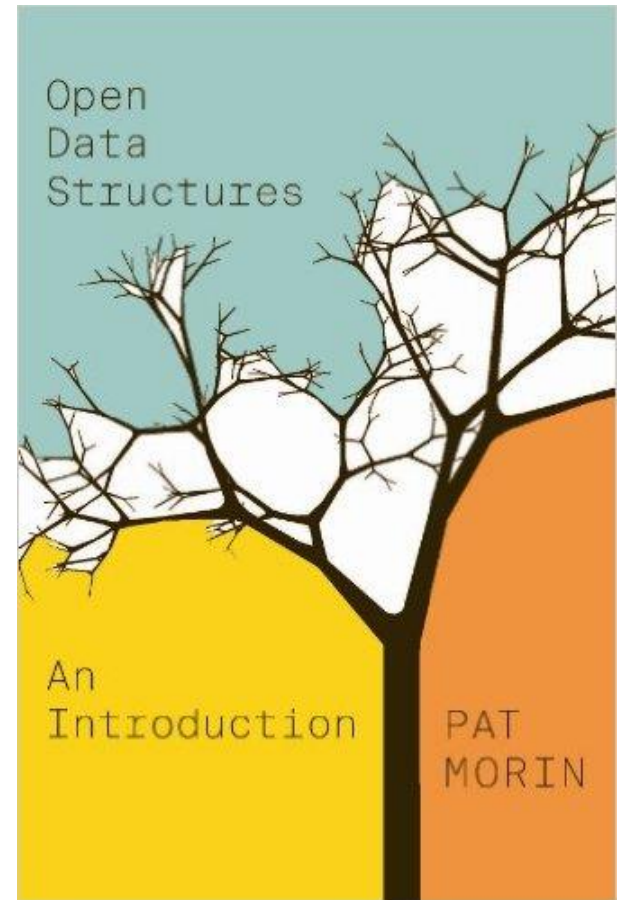
Abstract Data Types & Algorithms

# Readings

This class

- ods 4

Skiplists



# Lists

We have seen array based and linked-list based implementations of the List interface now.

They each have their strengths and weaknesses.

You choose the data structure that best suits your application (data usage).

# Lists

| Operations                 | Arrays | Linked Lists |
|----------------------------|--------|--------------|
| get/set near the ends      |        |              |
| get/set near the middle    |        |              |
| add/remove near the ends   |        |              |
| add/remove near the middle |        |              |

# Lists

| Operations                 | Arrays      | Linked Lists       |
|----------------------------|-------------|--------------------|
| get/set near the ends      | <b>Fast</b> | <b>Fast</b>        |
| get/set near the middle    | <b>Fast</b> | <b>Slow</b>        |
| add/remove near the ends   | <b>Fast</b> | <b>Fast</b>        |
| add/remove near the middle | <b>Slow</b> | <b>Fast (Slow)</b> |

# Lists

| Operations                 | Arrays | Linked Lists      |
|----------------------------|--------|-------------------|
| get/set near the ends      | $O(1)$ | $O(1)$            |
| get/set near the middle    | $O(1)$ | $O(n)$            |
| add/remove near the ends   | $O(1)$ | $O(1)$            |
| add/remove near the middle | $O(n)$ | $O(1)$ ( $O(n)$ ) |

# Lists

Imagine a linked list in which you could do a binary search to find the index value.

But we'd need a bunch of pointers...

# Skiplist

A **skiplist** is a **randomized** data structure that implements the **List** interface supporting all of

`get(i)`, `set(i,x)`, `add(i,x)`, `remove(i)`

with **expected** runtime complexity  $O(\log n)$

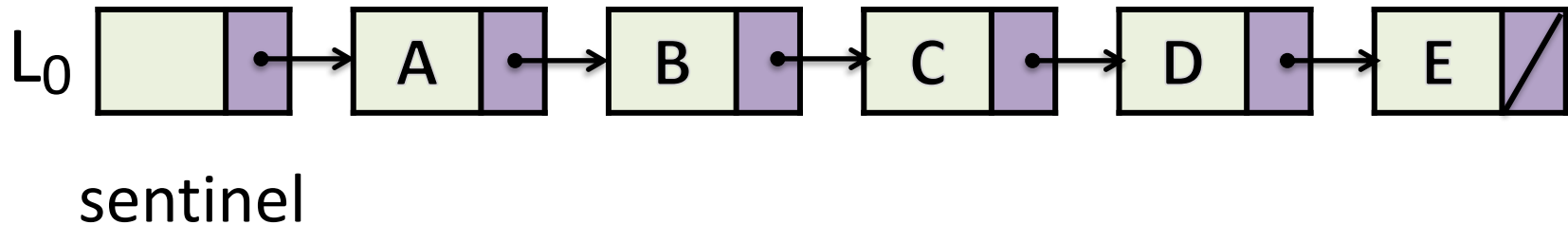
The data structure is built stochastically  
(using randomness)



# Skiplist

Underlying the **skiplist** is a singly-linked list  $L_0$

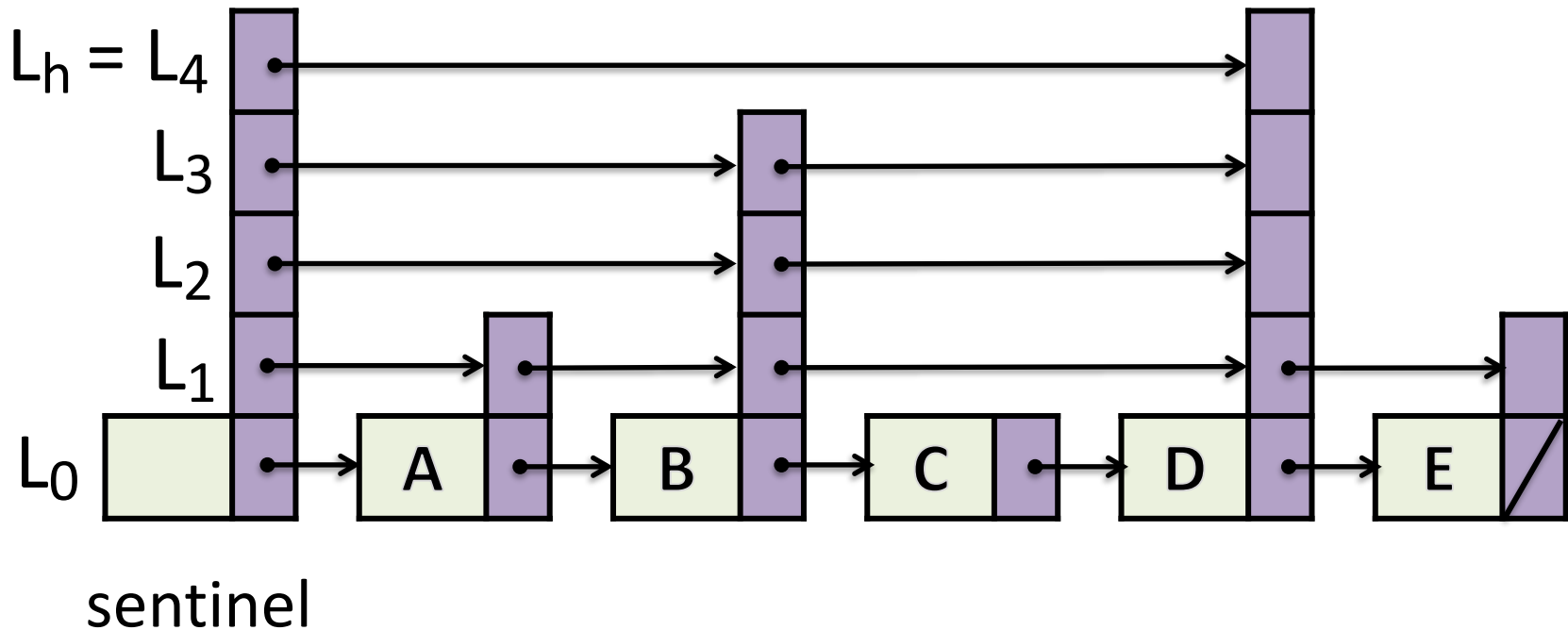
The list has a dummy node called the **sentinel**



# Skiplist

The **skiplist** is a **sequence** of singly-linked lists

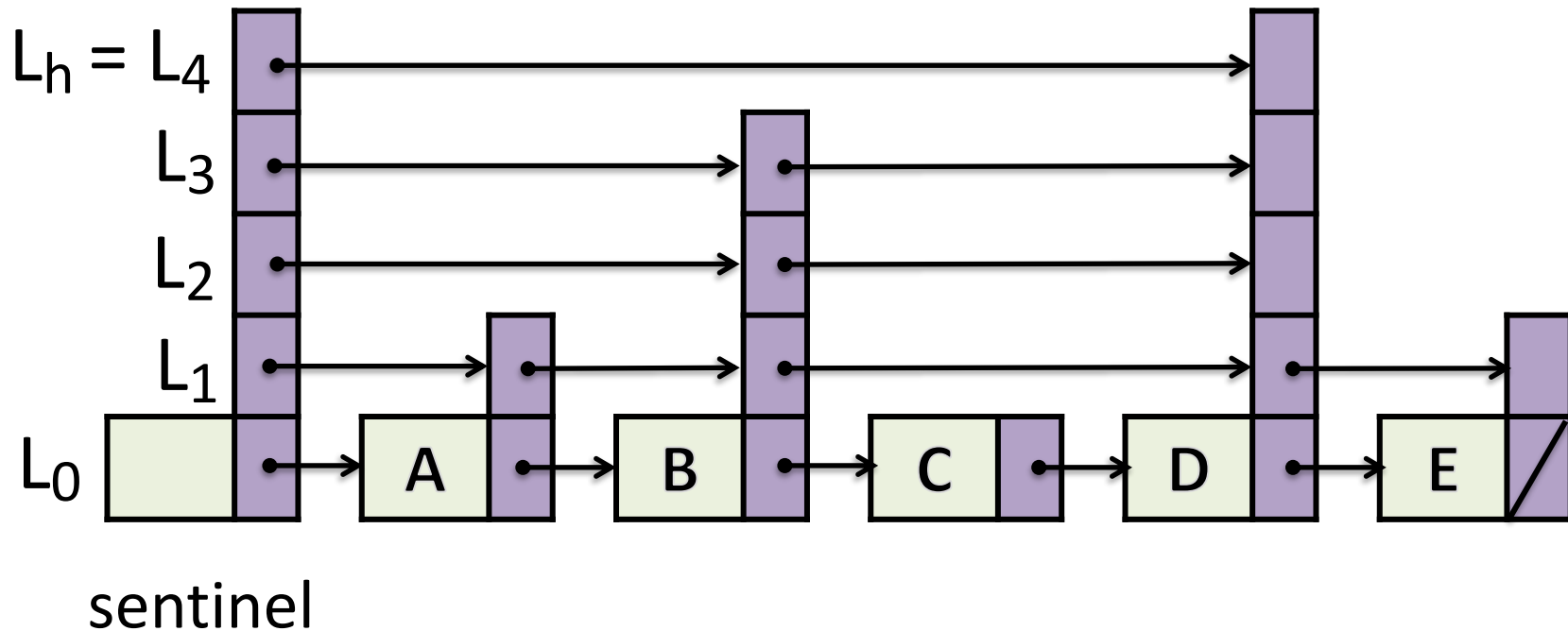
$$L_0, L_1, L_2, \dots, L_h$$



# Skiplist

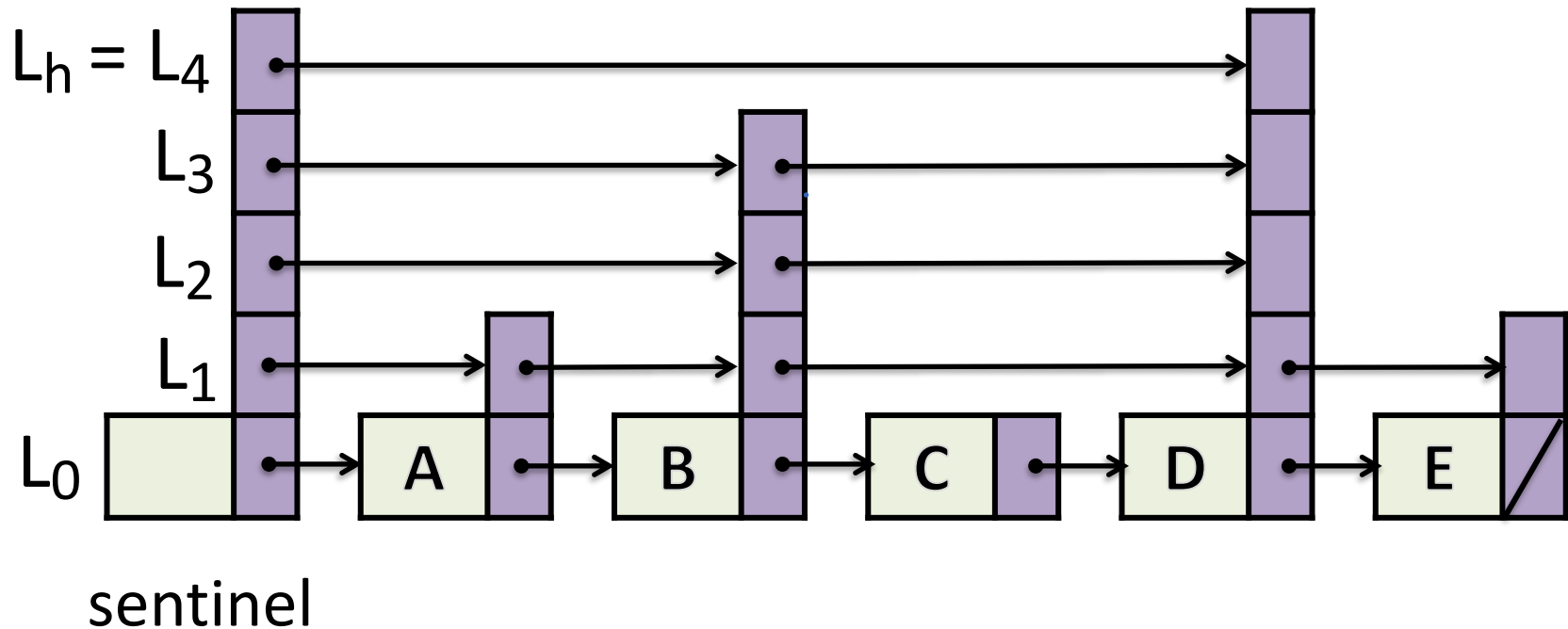
The **height** of the skiplist is  **$h$**

$$L_0, L_1, L_2, \dots, L_h$$



# Skiplist

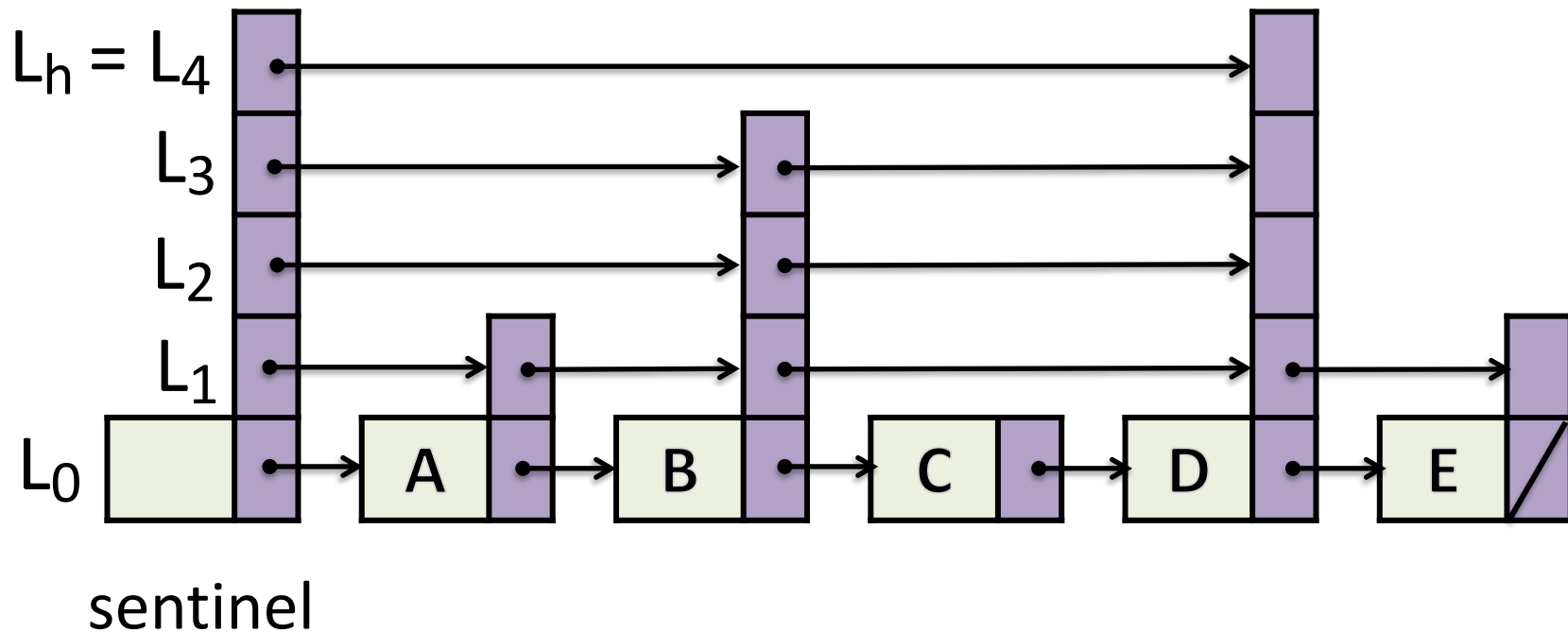
List  $L_k$  contains a subset of the elements of  $L_{k-1}$   
 $L_0, L_1, L_2, \dots, L_h$



# Constructing a Skiplist

Two ways to construct a skiplist

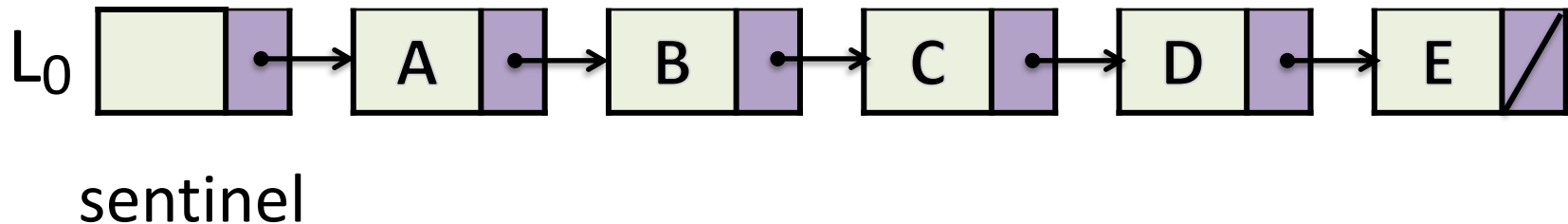
1. Convert existing linked list into a skiplist
2. Build it up one element at a time



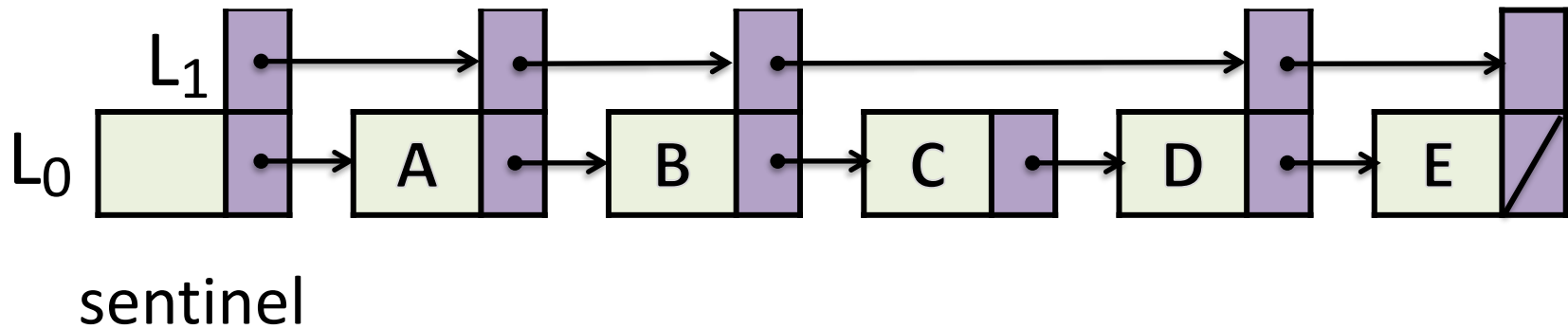
# Constructing a Skiplist

At a given level  $L_k$  flip a coin for element in the list to generate the level  $L_{k+1}$ .

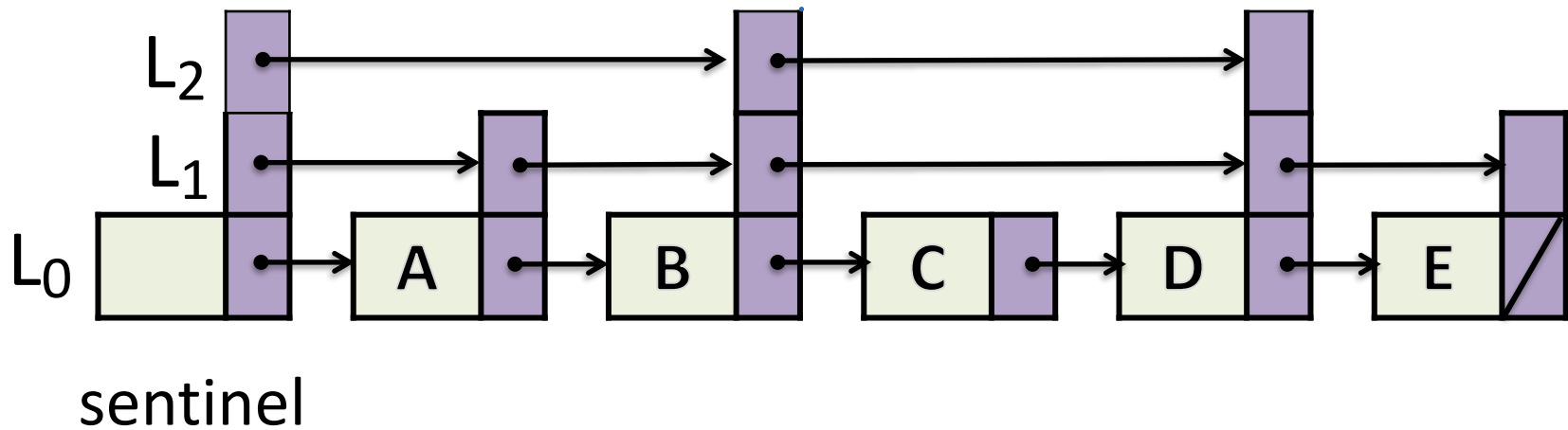
- Tails  $\rightarrow$  element is in next level
- Heads  $\rightarrow$  element is not in next level



# Constructing a Skiplist

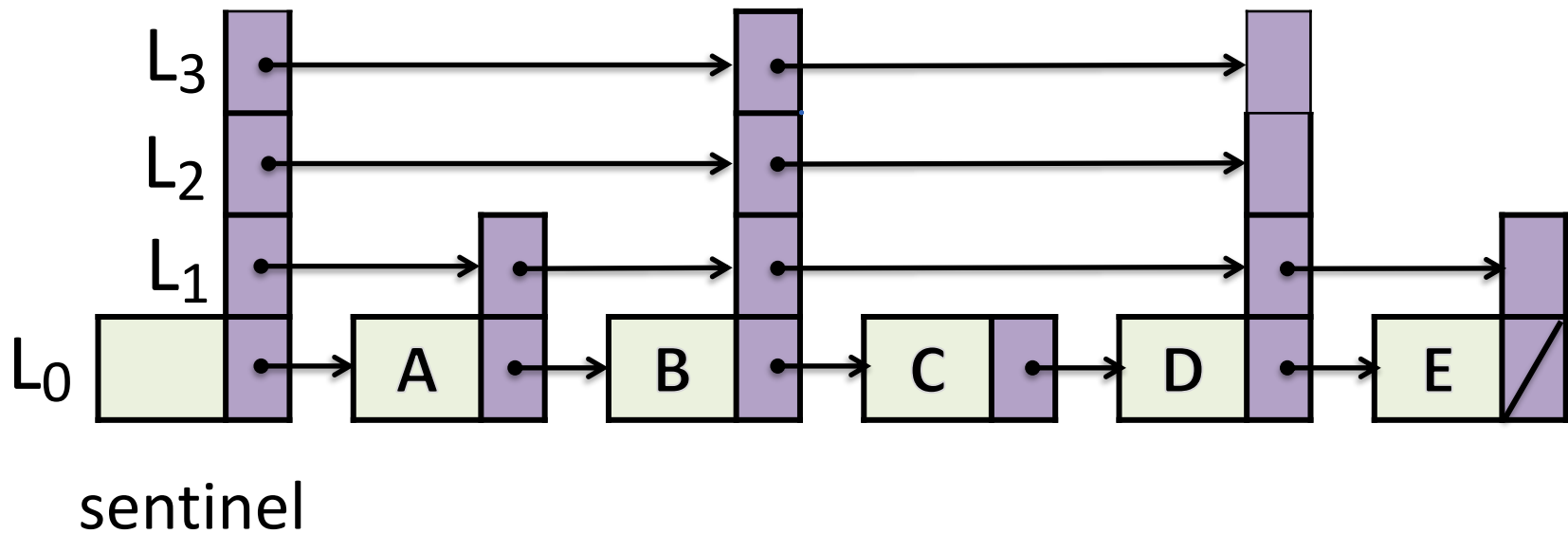


# Constructing a Skiplist

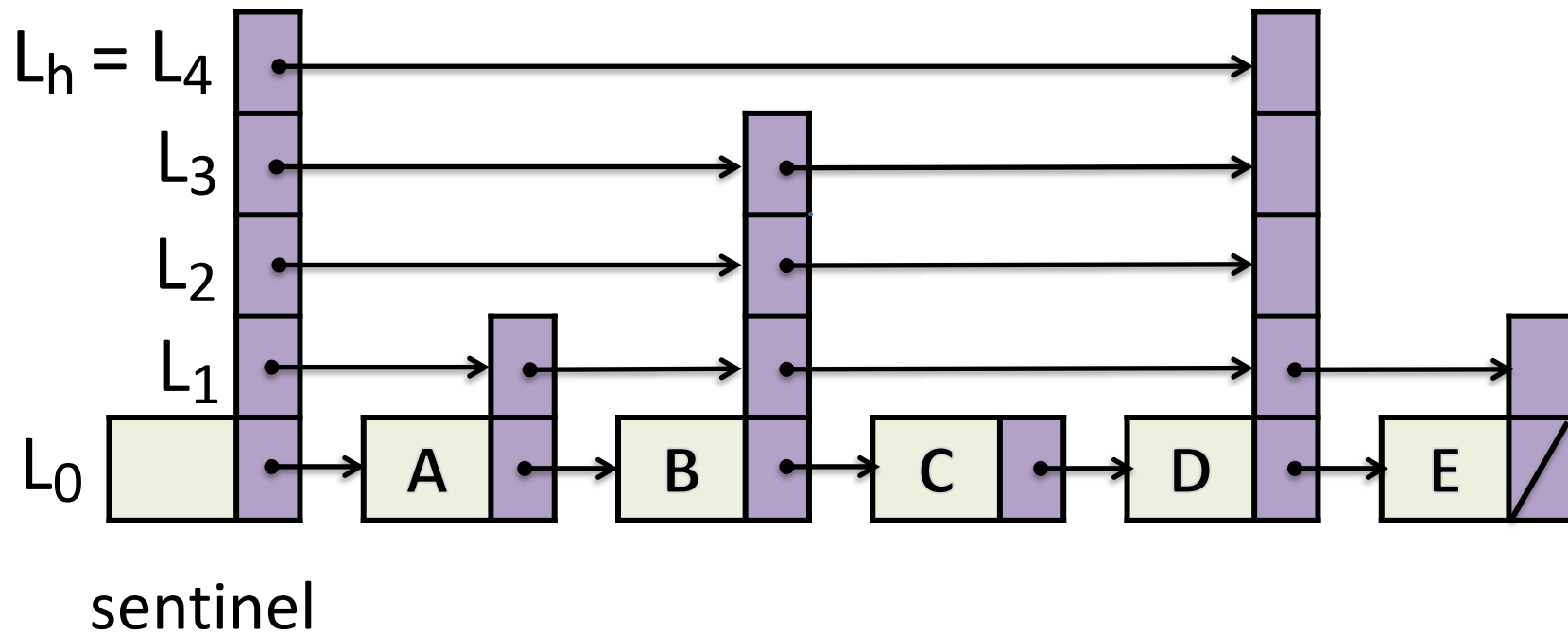




# Constructing a Skiplist



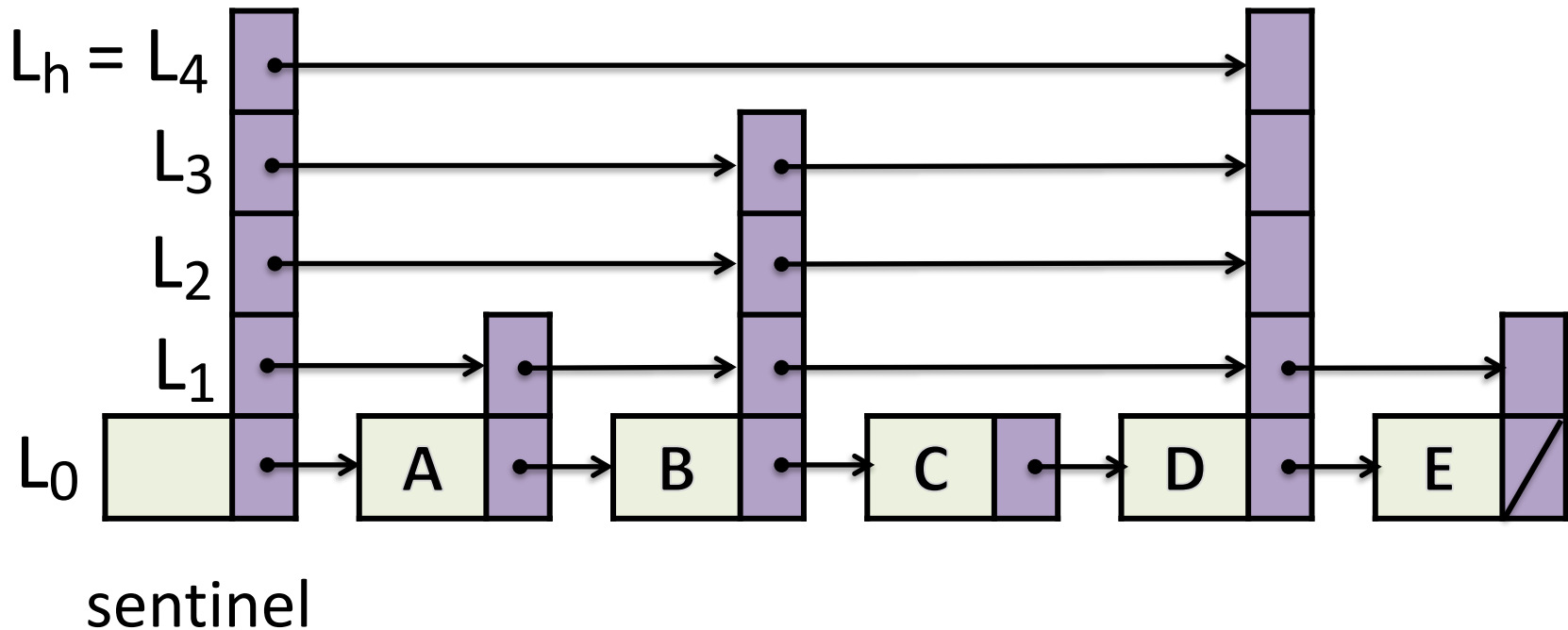
# Constructing a Skiplist



# Constructing a Skiplist

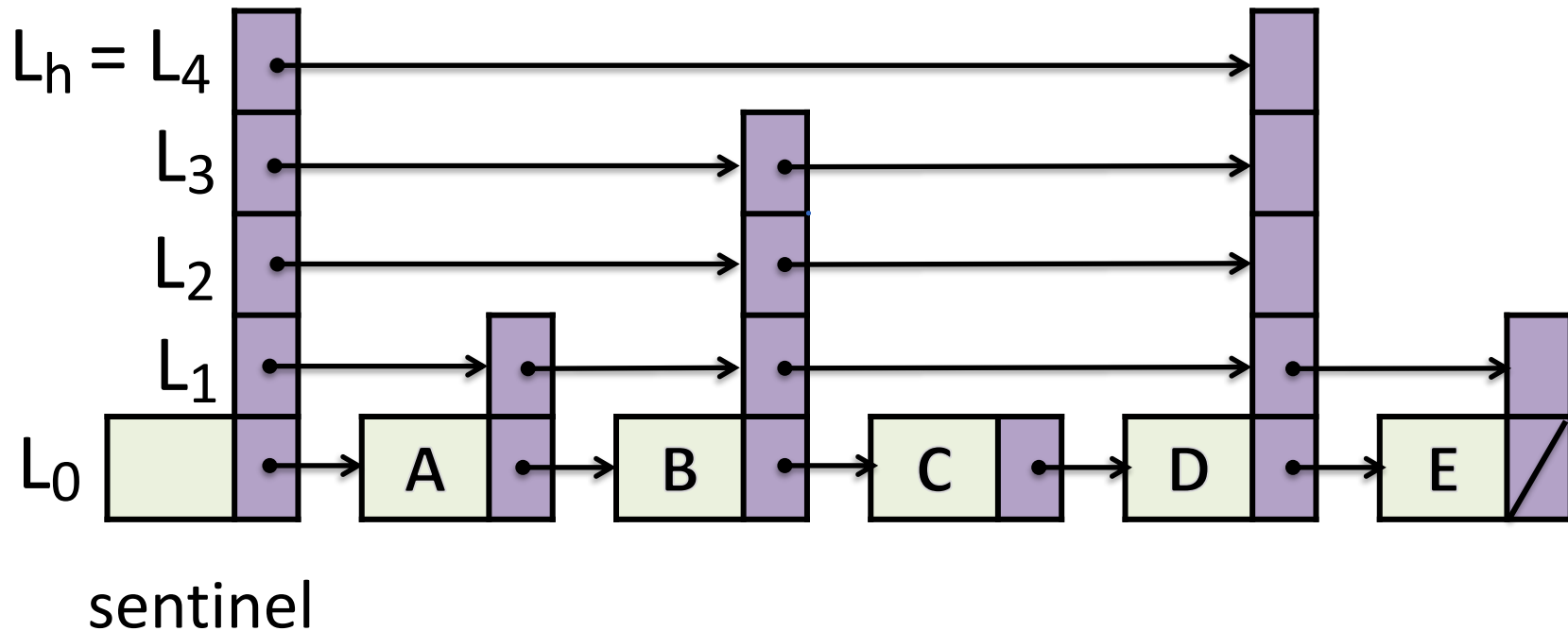
When adding a new element we determine its height by flipping a coin.

The height of a node is the number of tails that come up before a heads comes up.



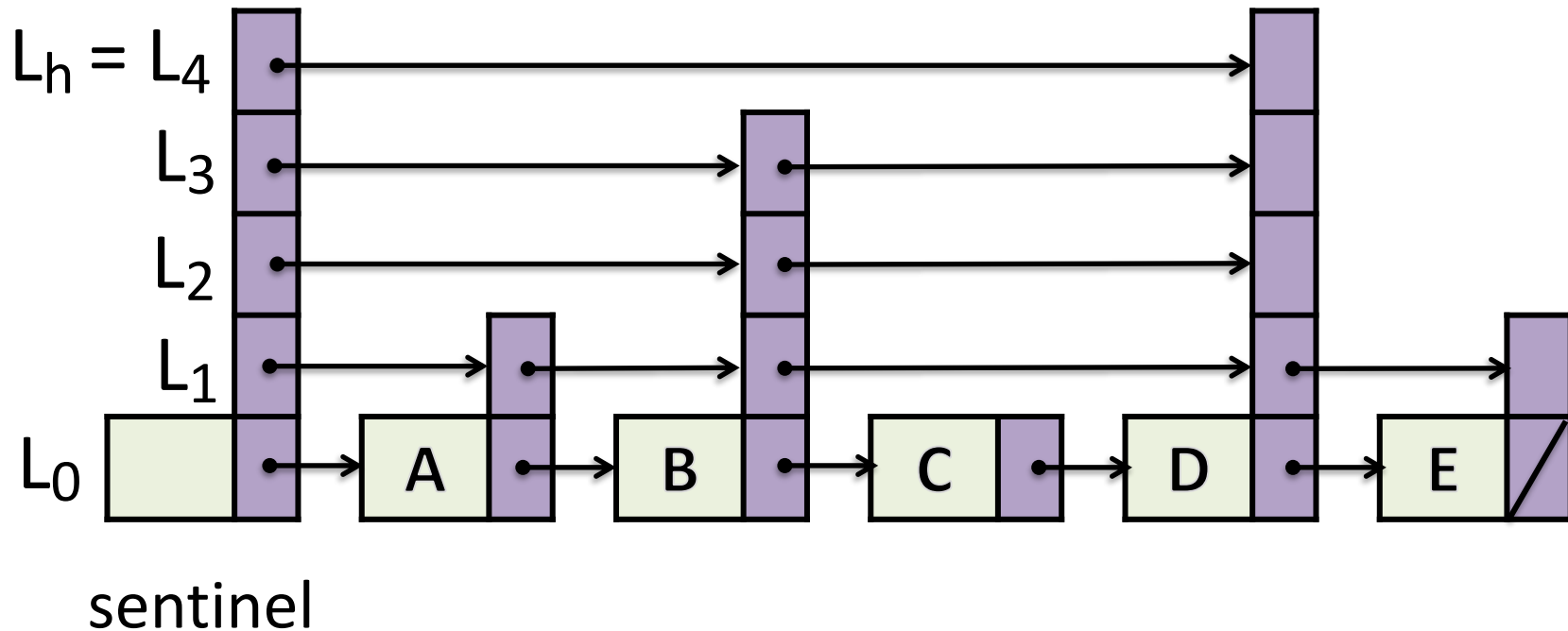
# Constructing a Skiplist

We use a pseudorandom bit generator to simulate a coin flip.



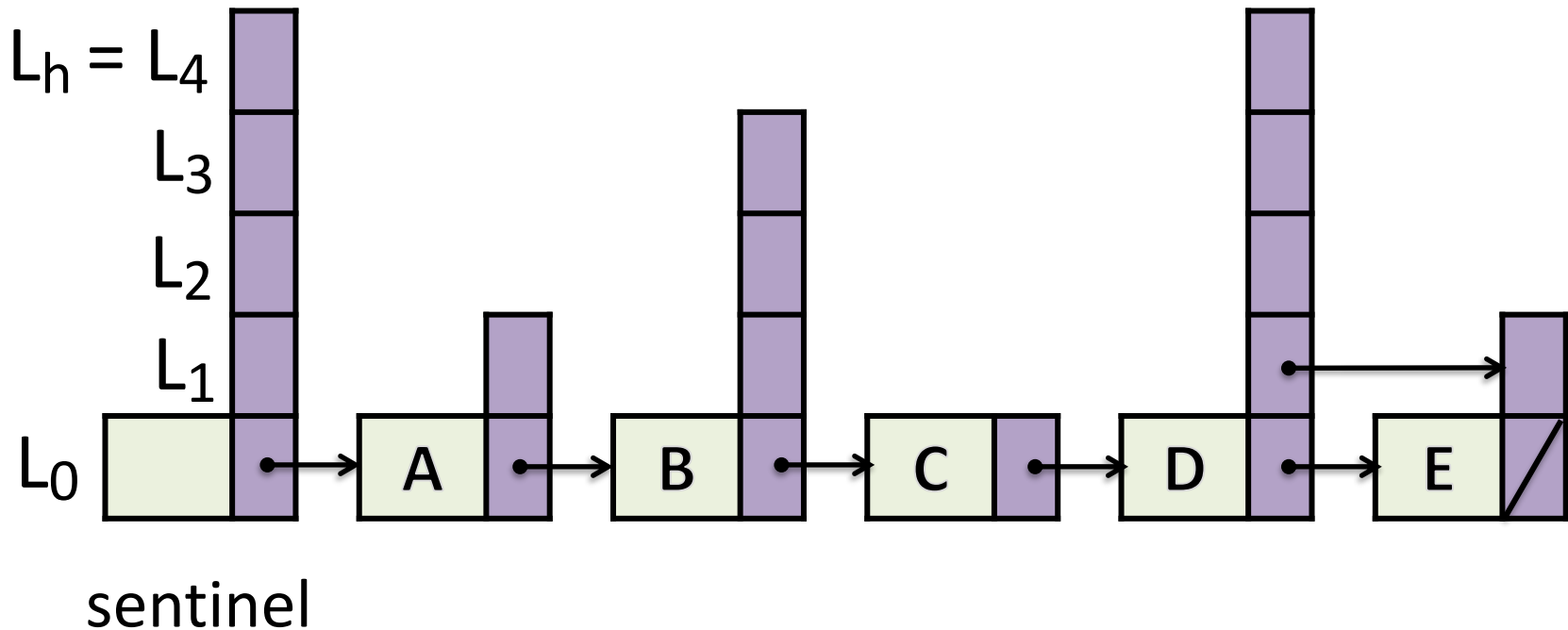
# Space Efficient Implementation

Each node contains a data value and an array of pointers called *next*.



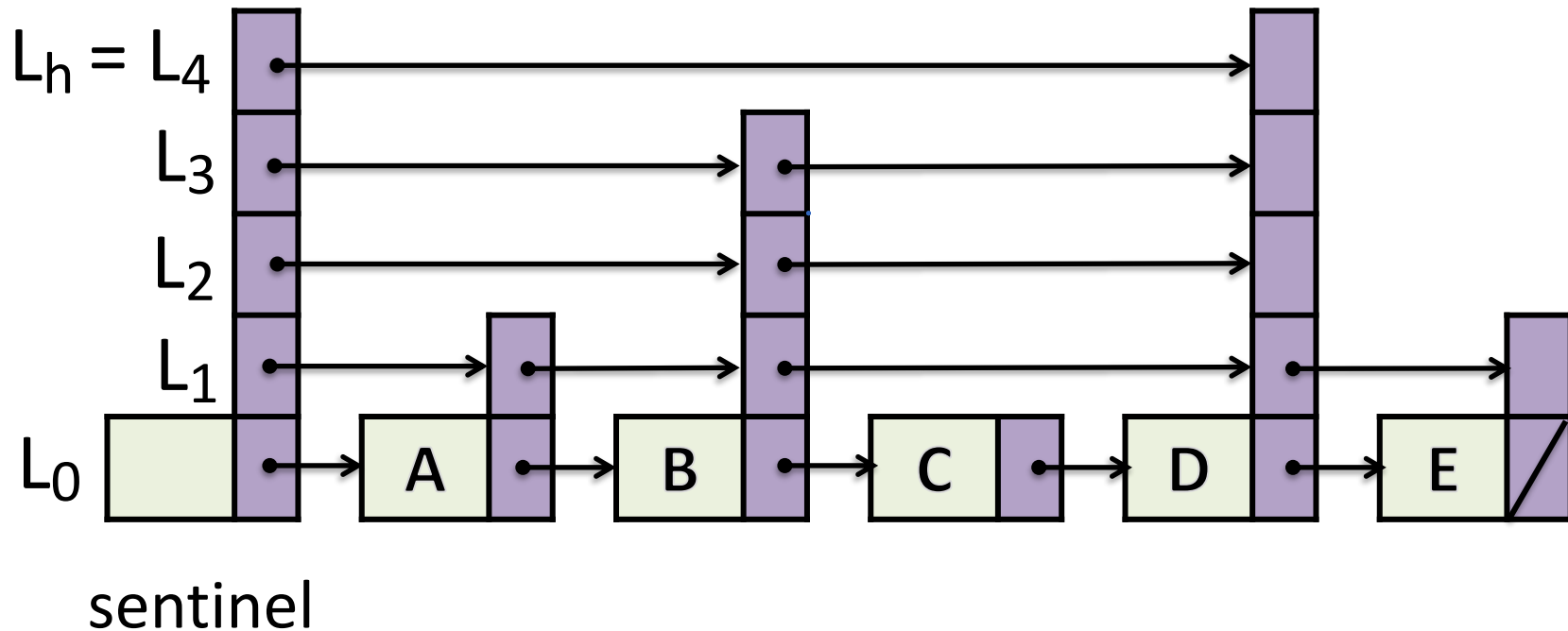
# Space Efficient Implementation

Each node contains a data value and an array of pointers called *next*.



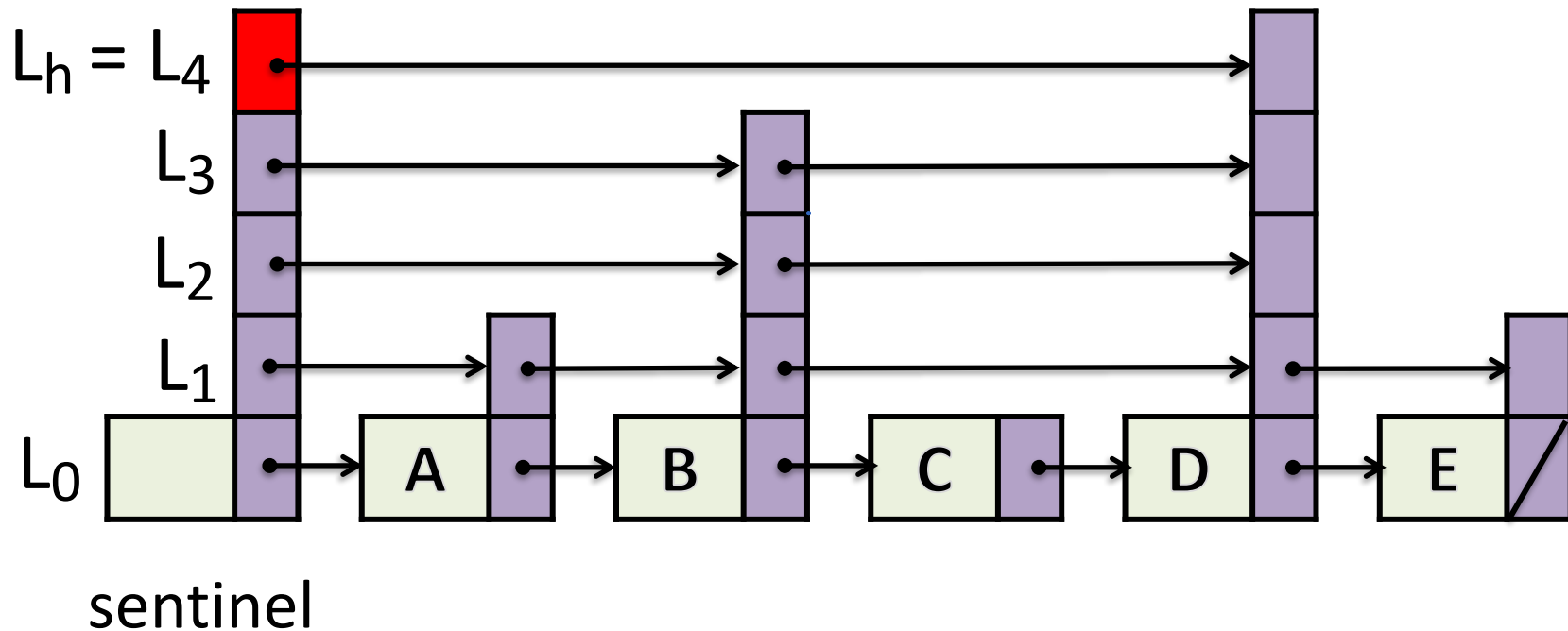
# Space Efficient Implementation

$u.next[k]$  points to  $u$ 's successor in  $L_k$



# The Search Path

There is a **short path**, called the **search path**, from the top of the sentinel to every node in the list ( $L_0$ )





# The Search Path

There is a **short path**, called the **search path**, from the top of the sentinel to every node in the list ( $L_0$ )

Start at the top of the sentinel  $L_h$ . Let  $i = h$

1. Move right in  $L_i$  until the next step would overshoot the target or you've reached the end of  $L_i$
2. Let  $i = i - 1$  (move down one level) and repeat step 1

# The Search Path

What does overshoot the target mean?

It depends on what interface the skiplist is implementing.

Section 4.2: SkiplistSSet (A sorted set)

Section 4.3: SkiplistList (Random-Access List)

## Lemma 4.1

The **expected** length of the search path for any node,  $u$ , in  $L_0$  is at most  $2 \log n + O(1)$ .

## Lemma 4.2

Let  $T$  be the number of times a fair coin is tossed up to and including the first time the coins come up heads. Then  $E[T] = 2$ .

Let  $N_T$  be the number of times a fair coin comes up tails before a heads comes up. Then  $E[N_T] = 1$ .

## Lemma 4.3

The **expected** number of nodes in a skiplist containing  $n$  elements, not including the sentinel, is  $2n$ .

## Lemma 4.4

The **expected** height of a skiplist containing  $n$  elements is at most  $\log n + 2$ .

## Lemma 4.4

The **expected** height of a skiplist containing  $n$  elements is at most  $\log n + 2$ .

Define the indicator variable  $I_r$ , for every positive integer  $r$ , to be

$$I_r = \begin{cases} 0 & \text{if } L_r \text{ is empty} \\ 1 & \text{if } L_r \text{ is not empty} \end{cases}$$

## Lemma 4.5

The expected number of nodes in a skiplist containing  $n$  elements, including the sentinel, is at most  $2n + O(\log n)$ .



## Lemma 4.6

The **expected** length of the search path for any node,  $u \in L_0$ , is at most  $2 \log n + O(1)$ .

## Theorem 4.3

A skiplist containing  $n$  elements has **expected** size  $O(n)$  and the **expected** length of the search path for any particular element is at most  $2 \log n + O(1)$ .

# SkiplistSSet

A skiplist can implement the Sorted Set Interface

add(x) : adds x to sorted set **if not already present**

remove(x) : removes x from the set (**if x is in the set**)

find(x) : returns the smallest  $y \geq x$  in the set

All operations have **expected**  $O(\log n)$  time complexity

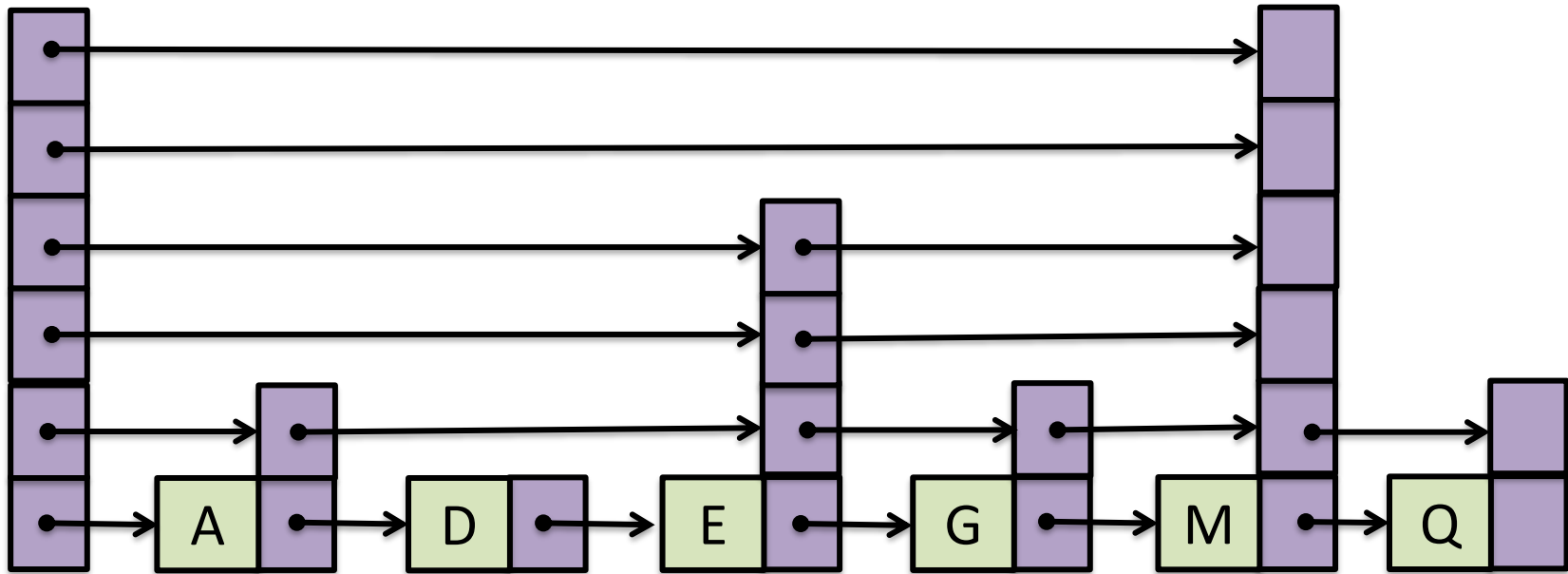
# SkiplistSSet

```
Node<T> findPredNode(T x) {  
    Node<T> u = sentinel;  
    int r = h;           // r is current level  
    while (r >= 0) {  
        while (u.next[r] != null && comp(u.next[r].x, x) < 0)  
            u = u.next[r]; // go right in list r  
        r--;               // go down into list r-1  
    }  
    return u;  
}
```

```
T find(T x)  
{ Node<T> u = findPredNode(x);  
  return u.next[0] == null ? null : u.next[0].x; }
```

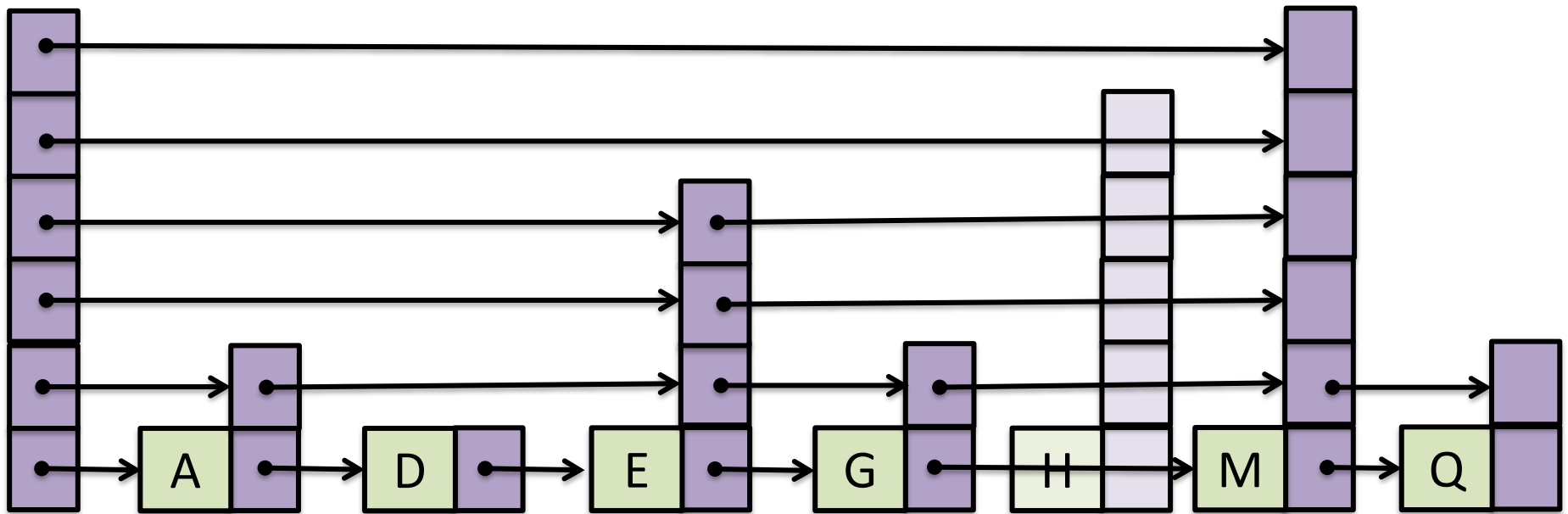
# Adding to SkiplistSSet

add(H) with height 4



# Adding to SkiplistSSet

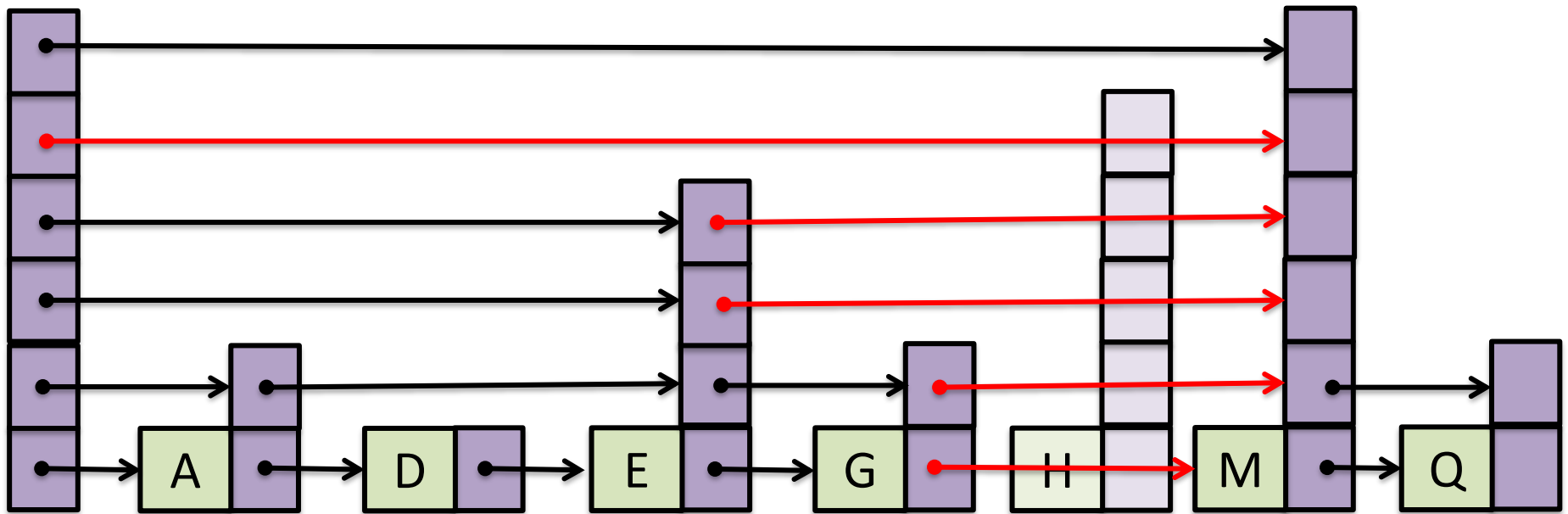
add(H)



# Adding to SkiplistSSet

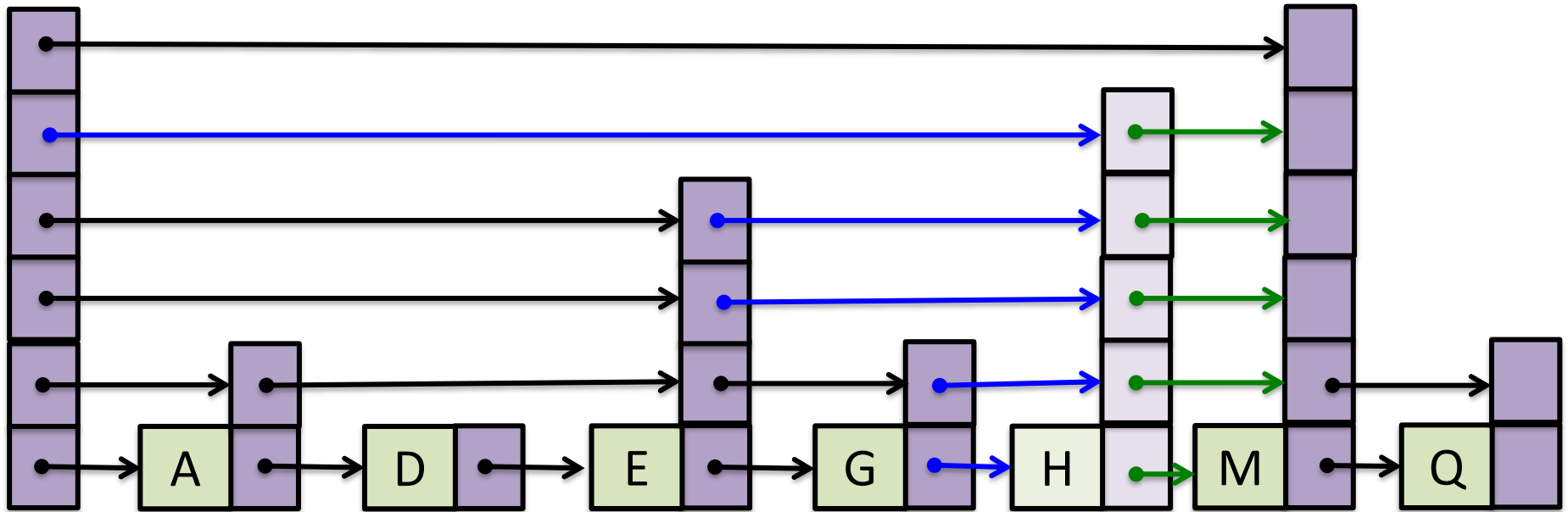
add(H)

We need to splice H into **some** lists  $L_k$



# Adding to SkiplistSSet

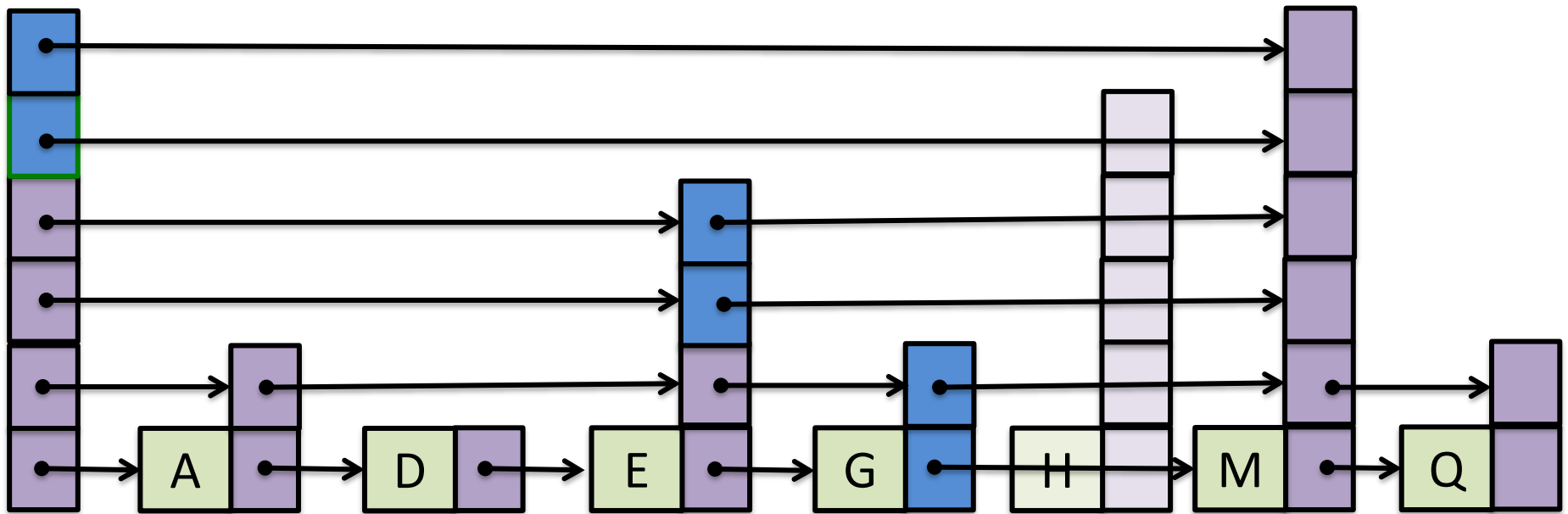
add(H)





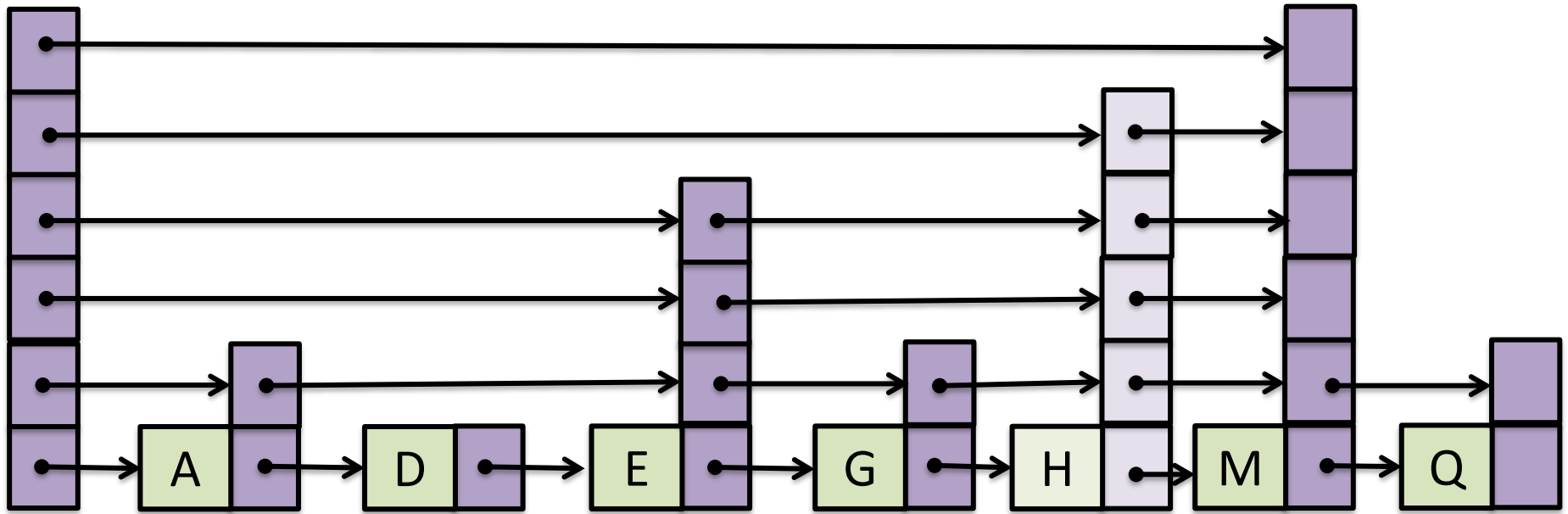
# Adding to SkiplistSSet

Keep track of right-most nodes of each level in the search path (in an array called **stack**)



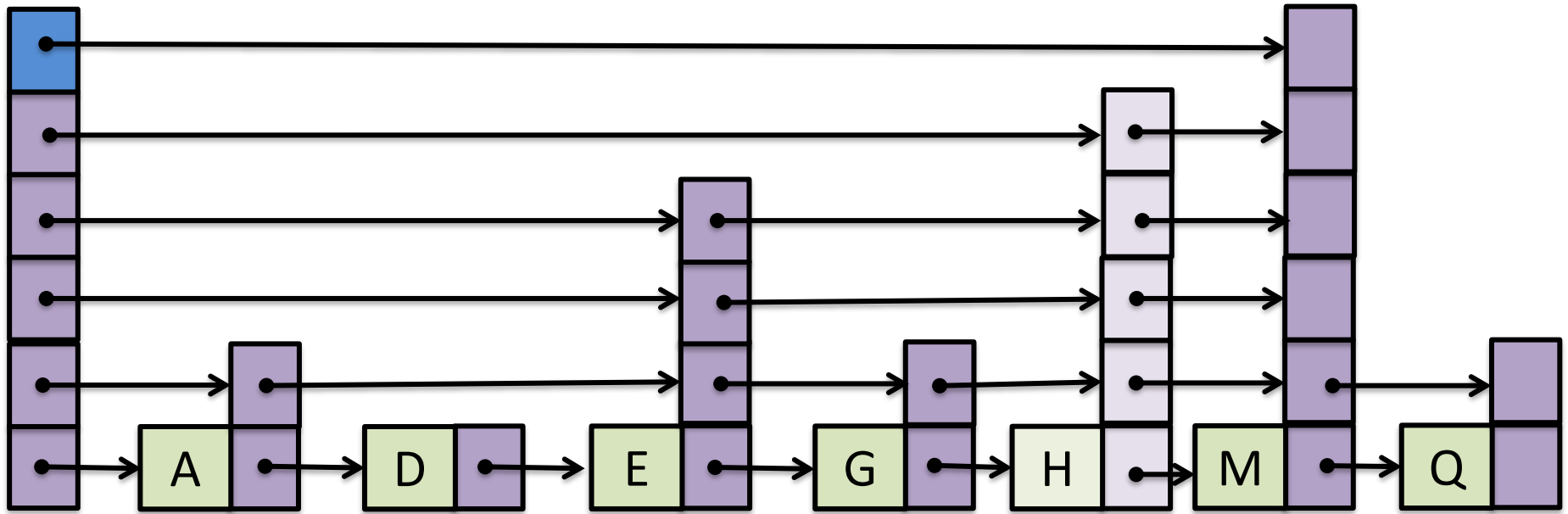
# Removing from SkiplistSSet

remove(H)



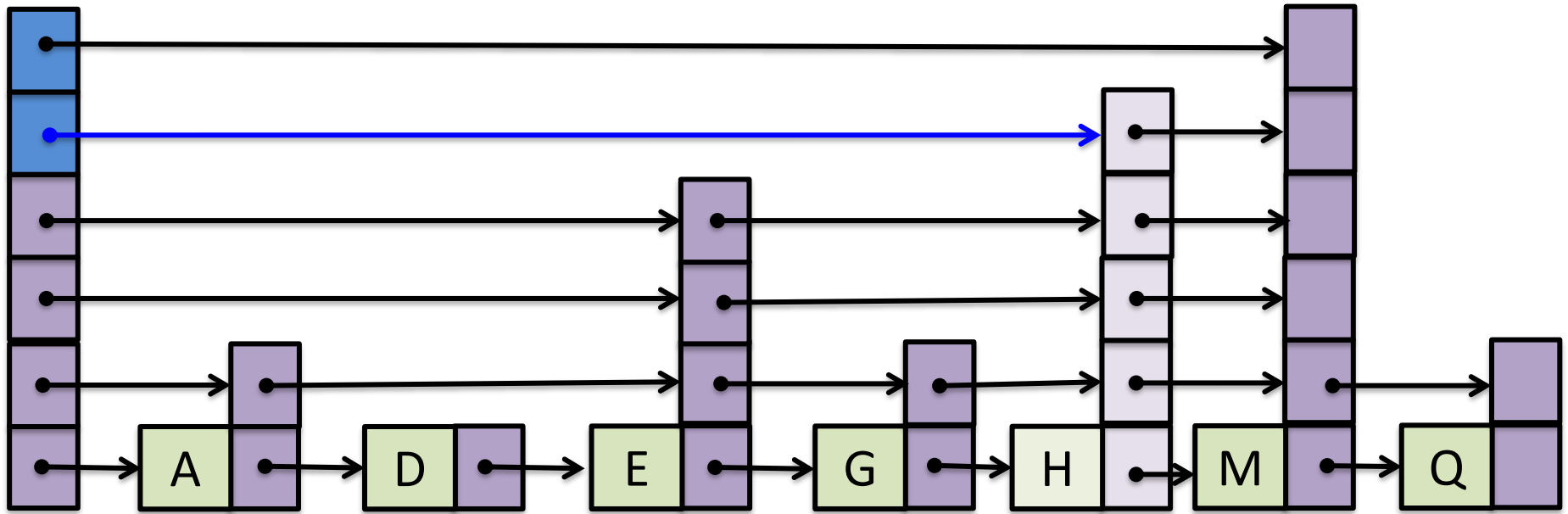
# Removing from SkiplistSet

remove(H)



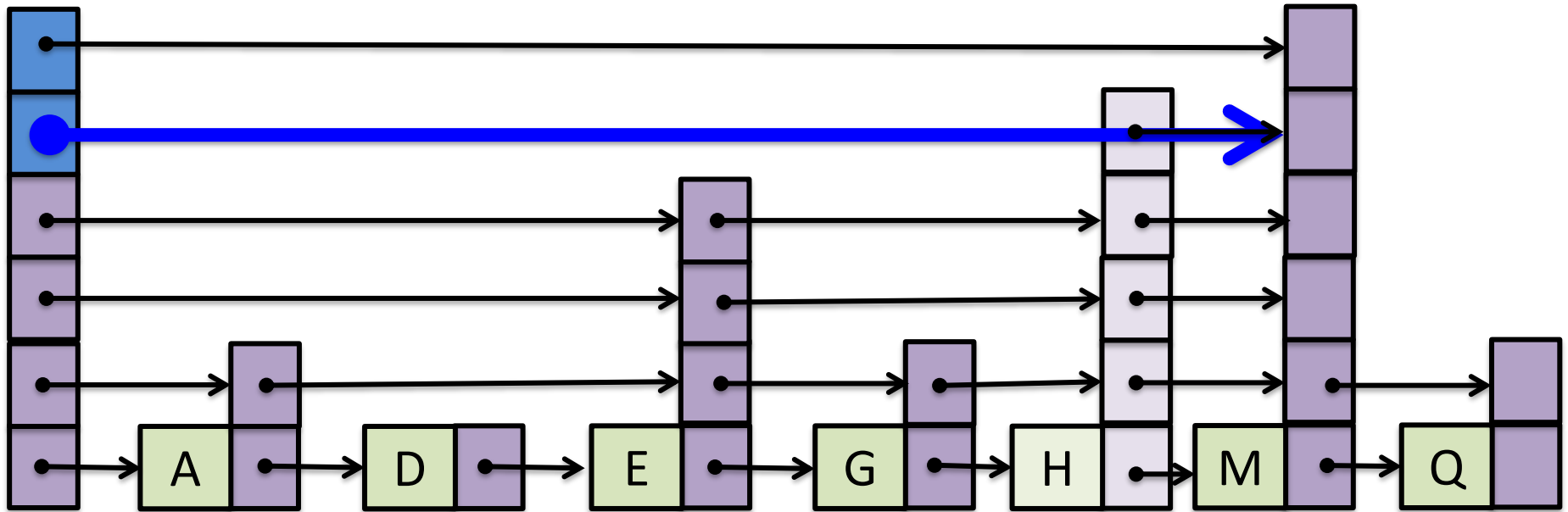
# Removing from SkiplistSet

remove(H)



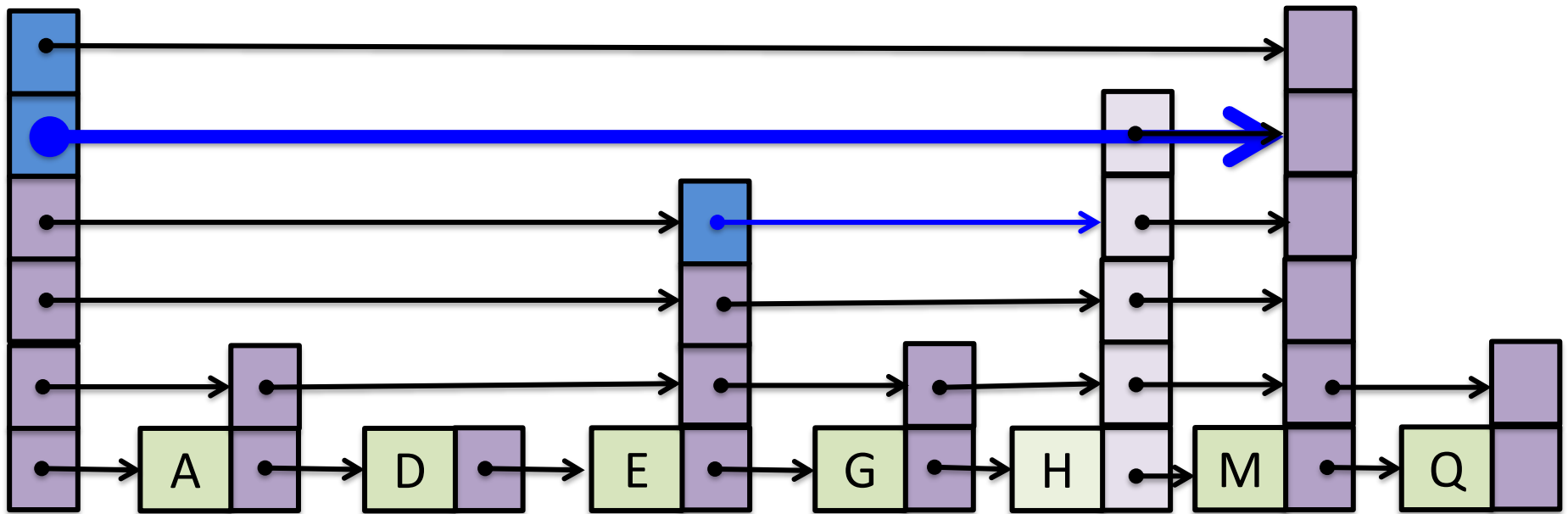
# Removing from SkiplistSet

remove(H)



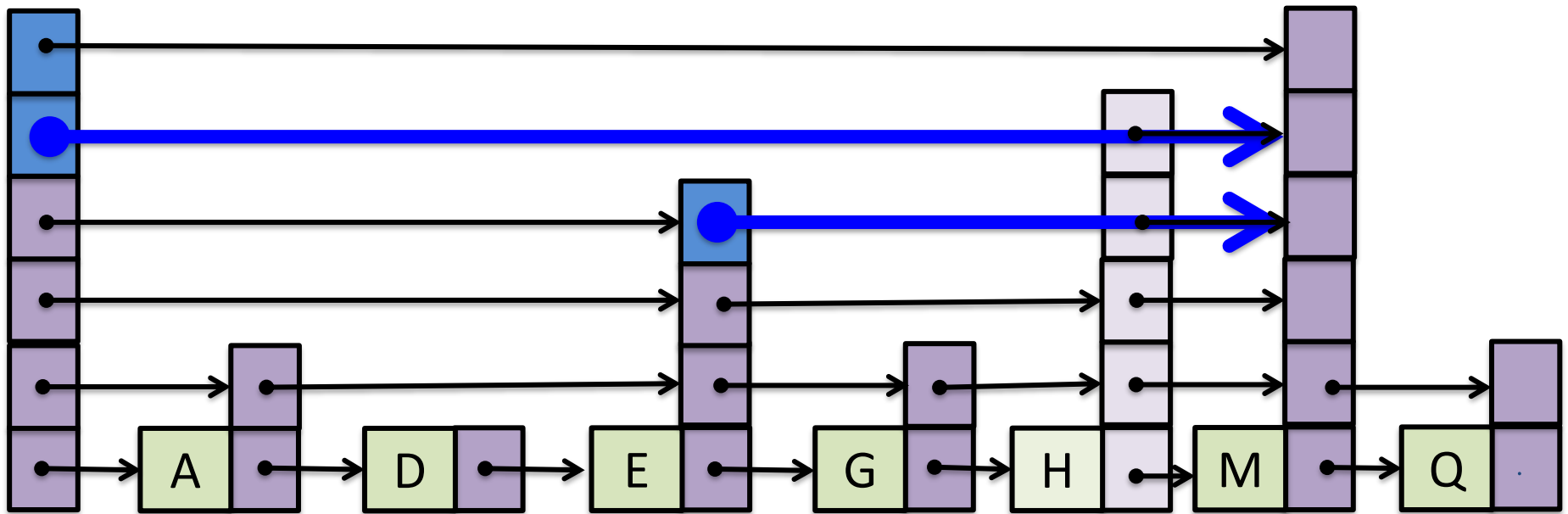
# Removing from SkiplistSet

remove(H)



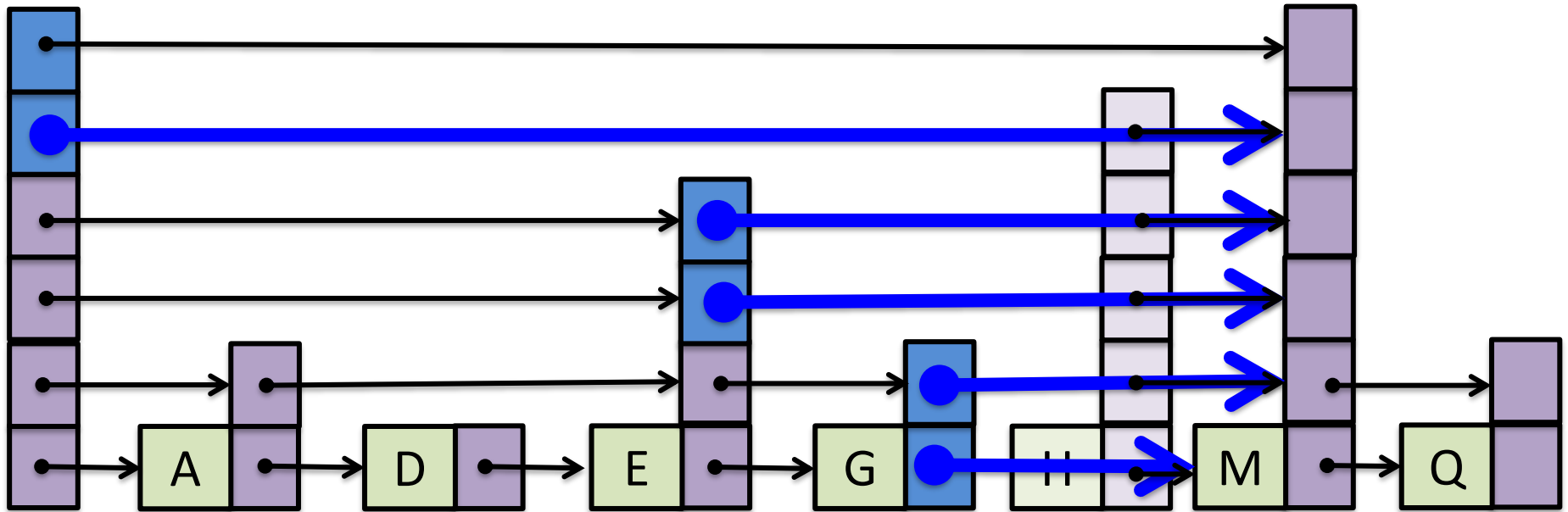
# Removing from SkiplistSet

remove(H)



# Removing from SkiplistSSet

remove(H)





# Removing from SkiplistSSet

remove(x)

- Follow the search path to x
- Along the way, if any node points to x, update its pointer to skip over x

# SkiplistSSet

`add(x)` : adds  $x$  to sorted set if not already present

`remove(x)` : removes  $x$  from the set (if  $x \in$  the set)

`find(x)` : returns the smallest  $y \geq x$  in the set

The cost of every operation is essentially the cost of finding a search path.

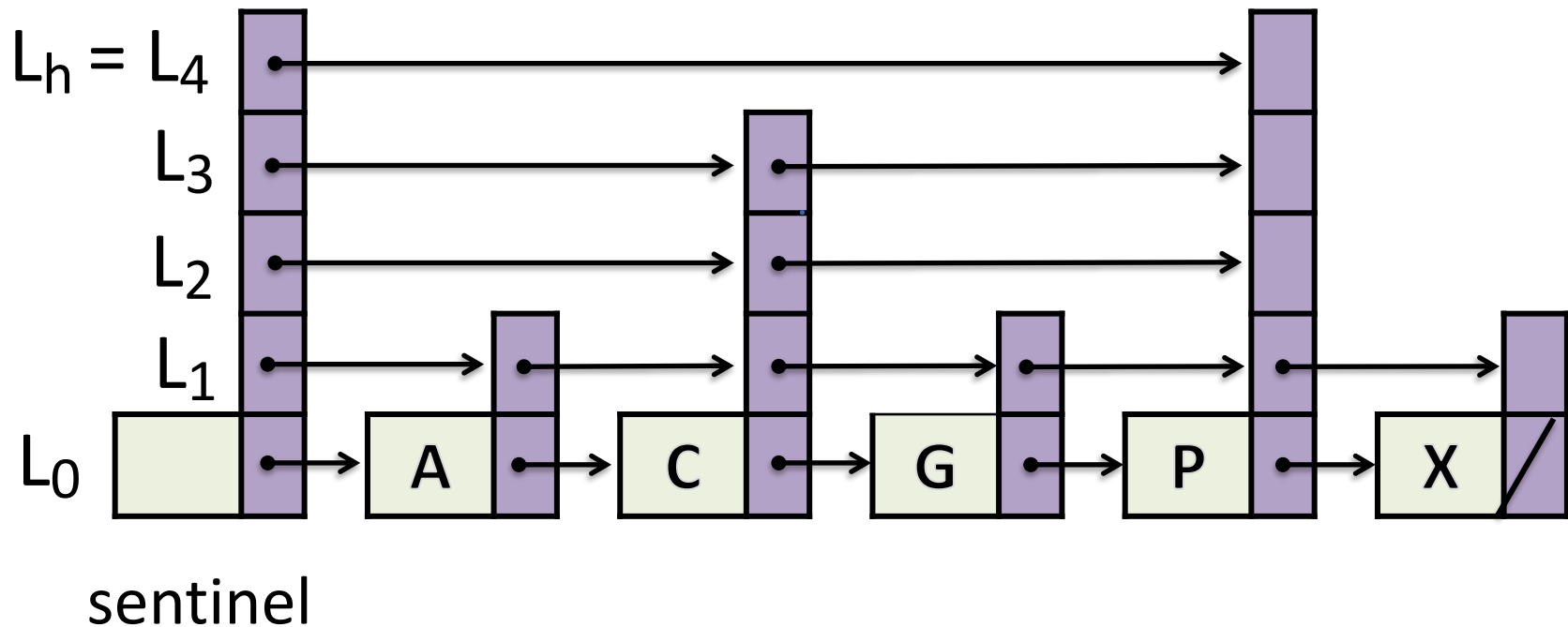
What is the **expected** length of a search path?

## Lemma 4.6

The expected length of the search path for any node,  $u \in L_0$ , is at most  $2 \log n + O(1)$ .

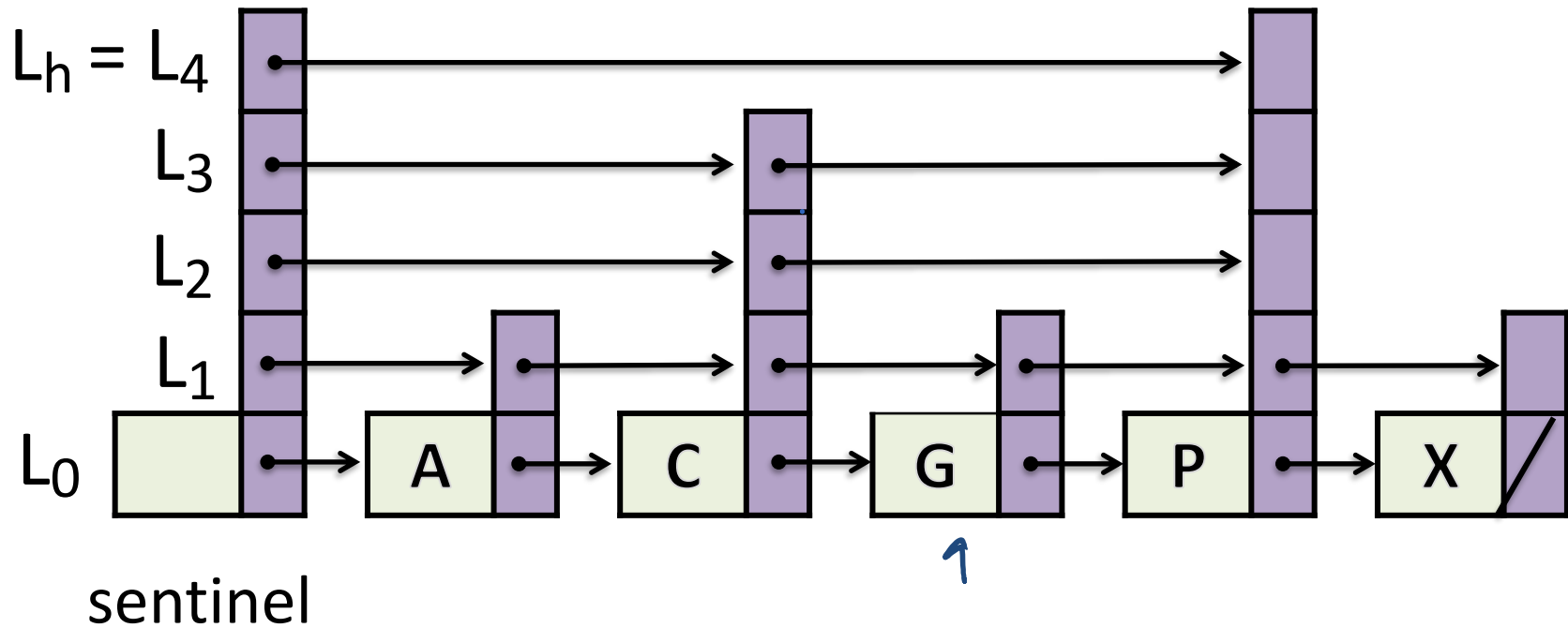
# Lemma 4.6

The expected length of the search path for any node,  $u \in L_0$ , is at most  $2 \log n + O(1)$ .



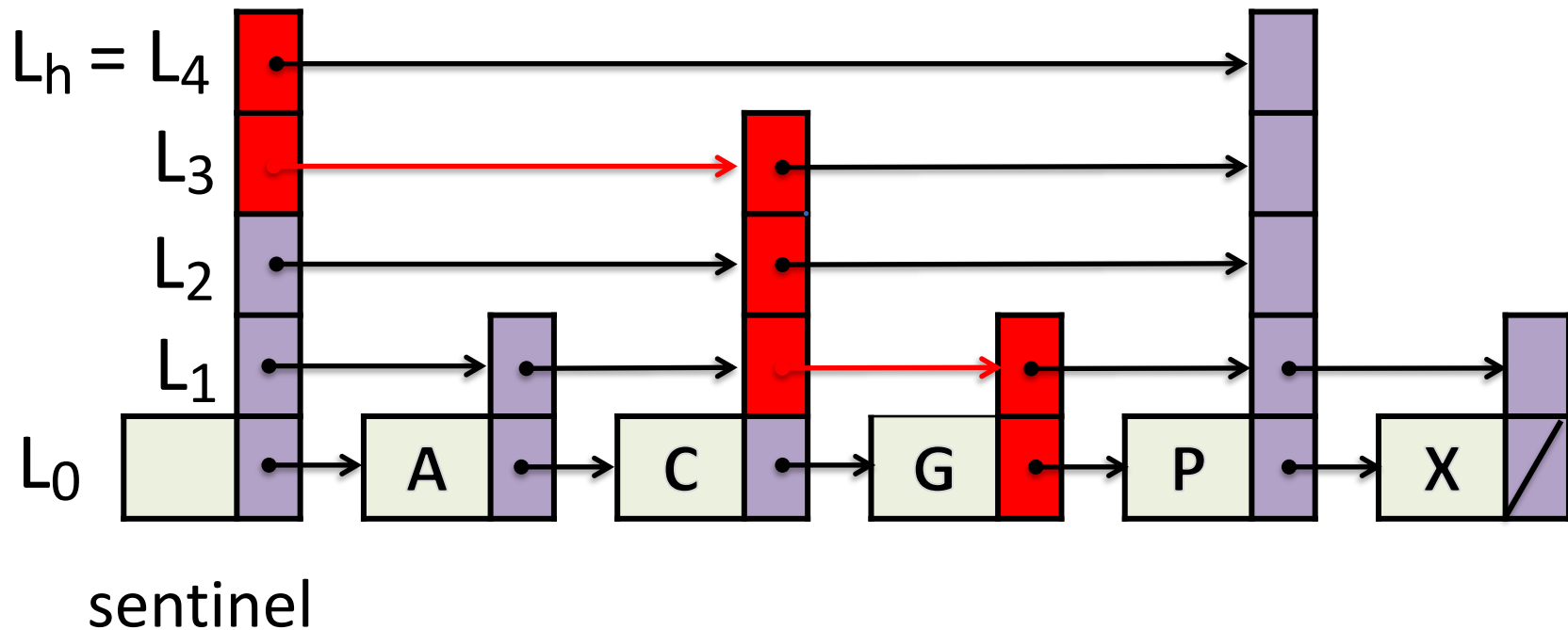
# Lemma 4.6

Search for target M (or P)



# Lemma 4.6

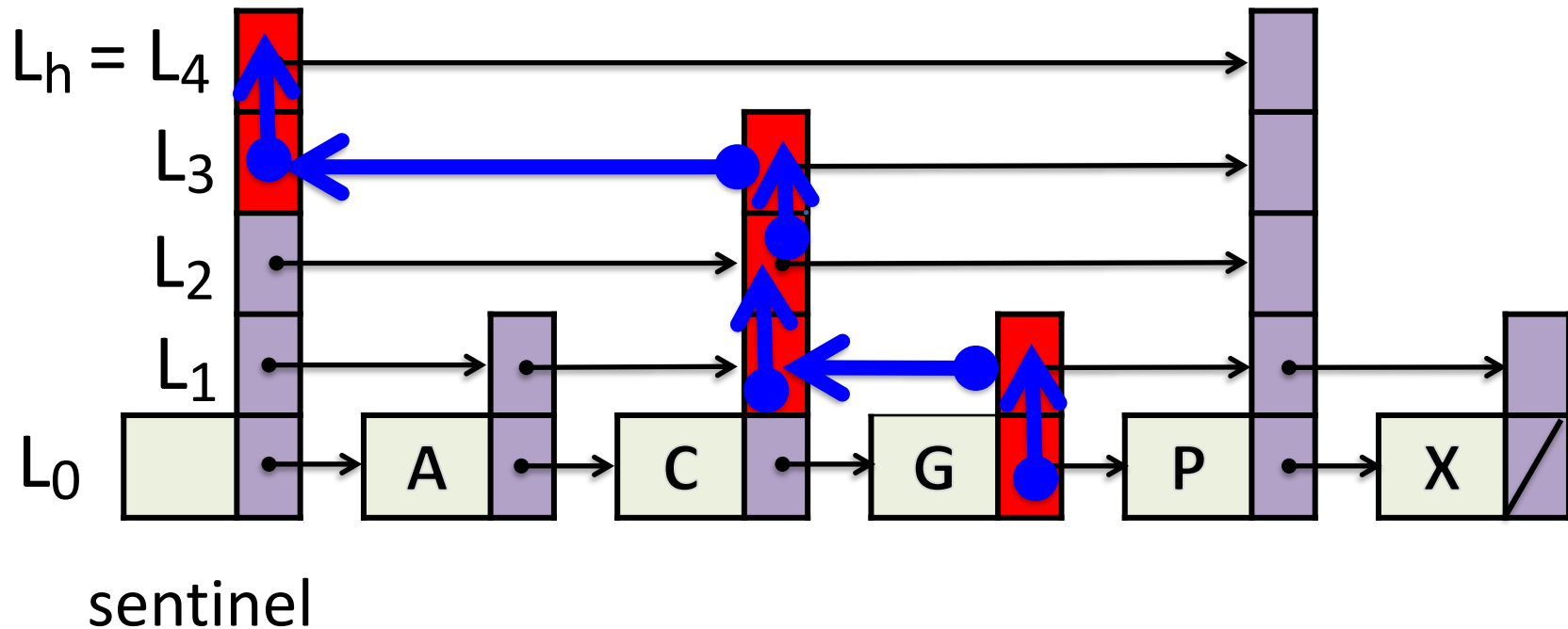
Search for target M (or P)



# Lemma 4.6

Consider the **reverse** search path.

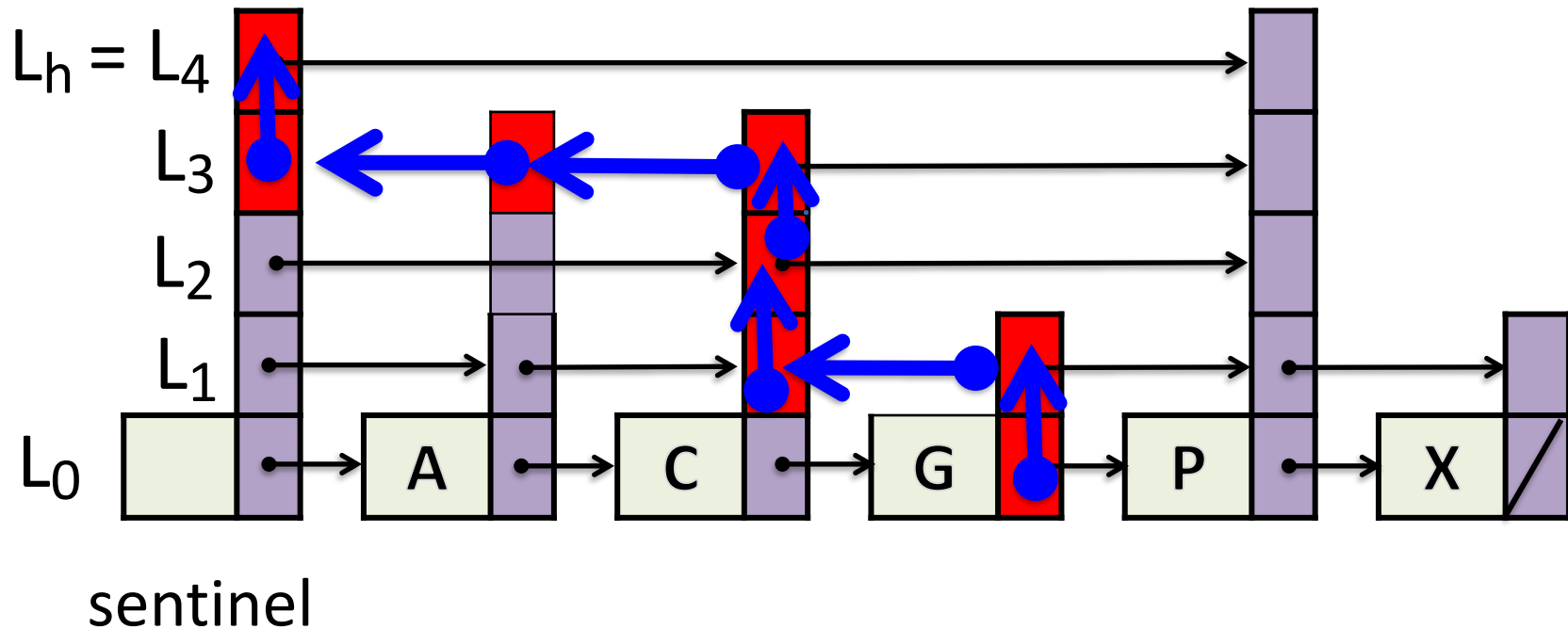
(move up if you can, otherwise move left. repeat)



# Lemma 4.6

Consider the **reverse** search path.

Note: there may be several left steps on a given level





## Lemma 4.6

The reverse path is determined as follows:

- At a given node, move up if we can.
- Otherwise, step left and then repeat.

Notice that the **reverse path** and the actual **search path** **are the same**.

(The reverse path is not an actual path of edges in the skiplist; it is a theoretical path)

## Lemma 4.6

The reverse path is determined as follows:

- At a given node, move up if we can.
- Otherwise, step left and then repeat.

We can simulate this as follows:

- At a given node, flip a coin and move up if heads.
- Move left if tails and then repeat.

# Lemma 4.6

We can simulate this as follows:

- At a given node, flip a coin and move up if heads.
- Move left if tails and then repeat.

What is the number of left steps on a given level?

What is the **expected** number of left steps on a given level?

## Lemma 4.6

What is the number of left steps on a given level?

What is the **expected** number of left steps on a given level?

Since reverse path and search path are the same, the expected number of left steps on a given level in the reverse path is also the expected number of right steps on the actual search path.

## Lemma 4.6

Let  $S_r$  denote the number right steps taken on level  $L_r$  in the search path (for some node)

From lemma 4.2, we then know that

$$E[S_r] = 1$$

Since the number of steps cannot be more than the number of nodes in a level, we also know that

$$E[S_r] \leq E[|L_r|] = n/2^r$$

## Lemma 4.6

Let  $S$  denote the length of a search path for any element in the skiplist. It should be clear that

$$S = h + \sum_{r=0}^{\infty} S_r$$

Therefore,

$$E[S] = E[h] + E\left[\sum_{r=0}^{\infty} S_r\right]$$

# Lemma 4.6

Let  $S$  denote the length of a search path for any element in the skiplist

$$S = h + \sum_{r=0}^{\infty} S_r$$

Therefore,

$$E[S] = E[h] + E\left[\sum_{r=0}^{\infty} S_r\right] \leq 2 \log n + 5$$

## Theroem 4.3

A skiplist containing  $n$  elements has expected size  $O(n)$  and the expected length of the search path for any element is at most  $2 \log n + O(1)$ .



# SkiplistList

A skiplist can implement a **random access list**

get(i) : returns  $x_i$

set(i,x) : sets  $x_i$  to be x

add(i,x) : inserts x as  $x_i$  and adjusts everything else

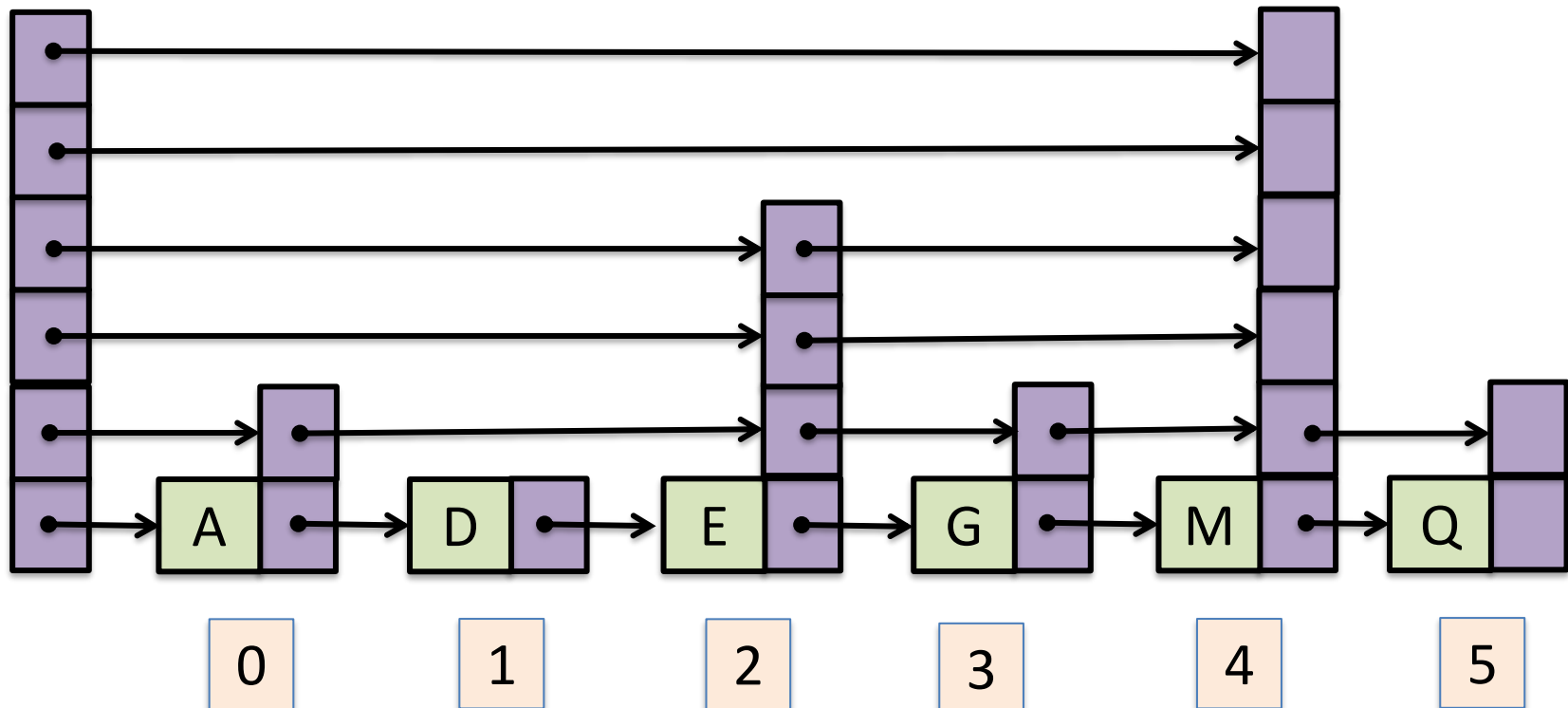
remove(i) : removes  $x_i$  from the list and adjusts the rest

All operations have time complexity  $O(\log n)$

(Be sure you read all of 4.3 for the details of this data structure)

# SkiplistList

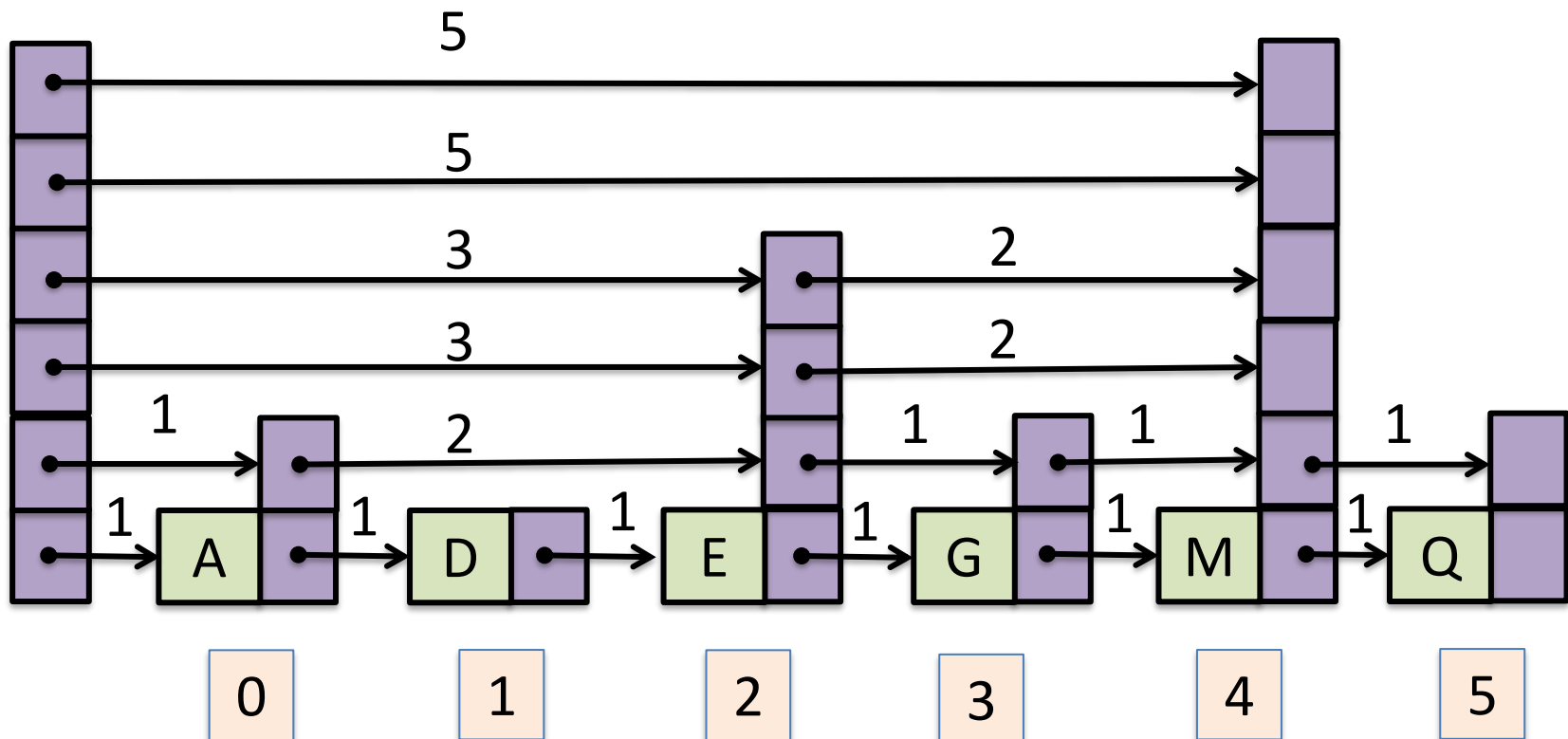
We need to be able to identify list indexes



# SkiplistList

Assign a **length** with each edge (pointer)

The value of an edge is the sum of all edge values in the level below it.



# SkiplistList

If we are at index  $j$  then following an edge with length  $m$  will bring us to index  $j + m$ . Let the sentinel have index -1.

